



OlegMos

13 часов назад

Как устроен RAG для юридических документов и почему одной LLM оказалось недостаточно

🔗 Средний ⌚ 9 мин 👁 5.1K

Python*, Искусственный интеллект, Data Engineering*, Natural Language Processing*, Машинное обучение*

Кейс

Из песочницы

Первый вариант системы выглядел вполне стандартно: документы разбиваются на фрагменты, для них считаются embeddings, пользователь задаёт вопрос, несколько наиболее подходящих фрагментов отправляются в LLM вместе с запросом. Модель формулирует ответ — на первых тестах этого было достаточно.

Проблемы начались на вопросах, которые на первый взгляд мало чем отличаются друг от друга:

- Какая пеня установлена в договоре № 14/24.7645?
- Кто из арендаторов не заплатил за май?
- А у него предусмотрена индексация?
- Контрагент предлагает добавить пеню в 0,01%. Насколько это существенно?

Формально всё это вопросы об аренде. Для системы это четыре разные задачи. В одном случае нужен точный поиск по конкретному документу, во втором — структурированные данные, в третьем надо понять, кого пользователь имеет в виду, а в четвёртом одного поиска вообще недостаточно: найденное условие ещё нужно оценить.

Я подошёл к этой задаче не как ML-инженер, а как юрист, много лет работавший с договорами. Поэтому довольно быстро выяснилось, что для меня важнее не то, насколько убедительно модель умеет отвечать, а можно ли понять, откуда появился вывод, повторится ли он на тех же данных и что система сделает в ситуации, которой не знает. Так первоначальный RAG над документами постепенно уступил место системе, способной по-разному обрабатывать разные типы запросов.

Где одного векторного поиска стало мало

Начальная схема была простой:

Документы → разбиение → embeddings → vector search → top-k → LLM

Объяснить с 📄

Для смысловых вопросов она работает хорошо.

В договоре, например, может быть написано:

Размер арендной платы может быть пересмотрен сторонами не чаще одного раза в год.

А пользователь спрашивает:

Может ли собственник каждый месяц повышать аренду?

Формулировки разные, смысл близкий. Здесь семантический поиск делает именно то, что от него требуется. Но договоры состоят не только из смыслов. В них постоянно встречаются номера,

фамилии, даты, реквизиты, статьи закона и другие значения, которые не нужно интерпретировать — их нужно точно найти.

Запрос договор 14/24.7645 сам по себе почти ничего не означает семантически. То же самое происходит с фамилией или номером статьи. Векторный поиск может решить, что содержательно похожий пункт другого договора релевантнее, хотя пользователю нужен конкретный документ.

Поэтому выбирать между лексическим и семантическим поиском я не стал. В текущем варианте запрос идёт сразу по двум веткам: BM25 отвечает за точные совпадения, BGE-M3 — за смысловую близость. Результаты объединяются через Reciprocal Rank Fusion.

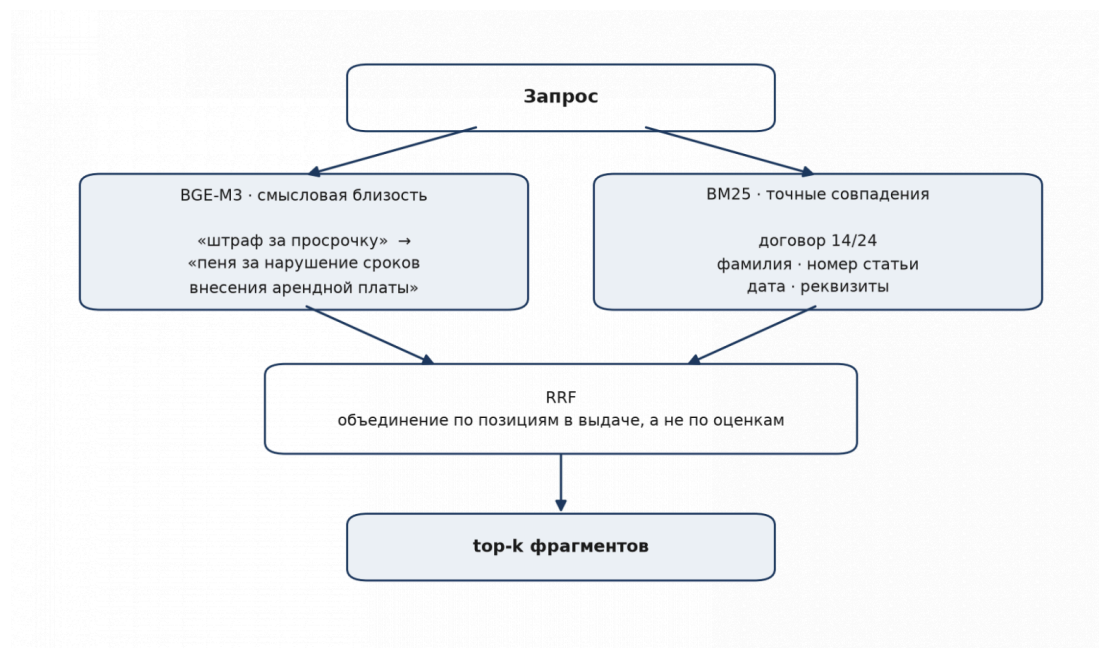


Схема гибридного поиска: BM25 + BGE-M3 → RRF → top-k

RRF здесь удобен тем, что не требует напрямую сравнивать оценки двух разных поисковых систем. У BM25 одна шкала, у векторного поиска другая. При объединении можно учитывать положение документа в каждой выдаче. На корпусе из **401 документа** итоговый Recall@10 сейчас составляет **0,935**.

Для проверки я использовал фиксированный набор из 31 запроса. Для каждого запроса был заранее определён документ, который должен попасть в выдачу. Recall@10 показывал, в скольких случаях нужный документ оказался среди первых десяти результатов. Все варианты поиска сравнивались на одном и том же наборе запросов.

Некоторое время именно эта метрика казалась мне главным ответом на вопрос, стал ли поиск лучше. Позже выяснилось, что высокий показатель ещё не говорит о реальном улучшении поиска.

Два правильных числа, из которых одно оказалось бесполезным

В какой-то момент у меня было два результата одного и того же показателя:

0,968 и **0,935**.

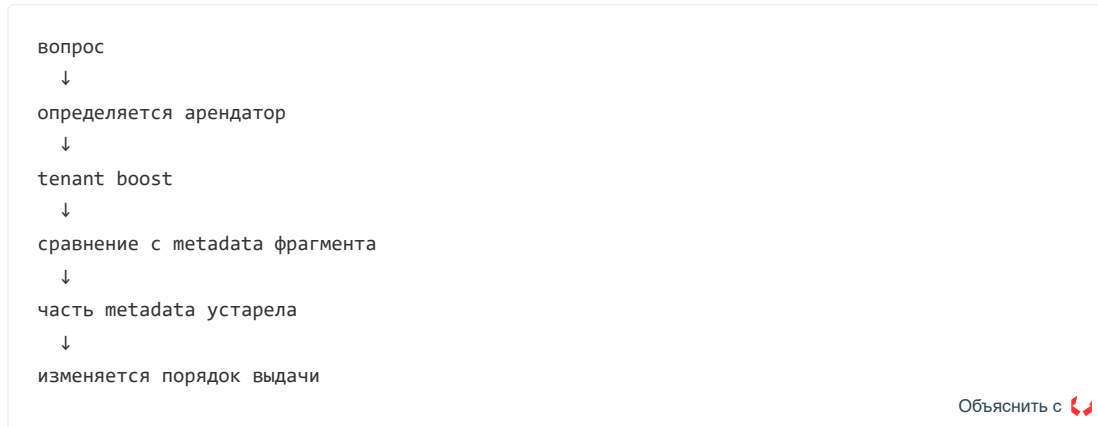
Оба были получены честным запуском evaluation. Ошибки в формуле не было. Но сравнивать их напрямую оказалось нельзя.

Результат 0,968 был получен, когда документы нужного арендатора дополнительно поднимались в выдаче с помощью механизма tenant boost. Если в вопросе определялся конкретный арендатор, фрагменты с соответствующим значением в metadata получали преимущество в выдаче. На первый взгляд логика правильная: если пользователь спрашивает про конкретного арендатора, документы этого арендатора действительно хочется поднять выше.

При отдельной проверке на девяти арендаторах механизм tenant boost не дал ни одного улучшения результата: **3 ухудшились, 6 остались без изменений**. Поэтому в рабочей конфигурации я его отключил. На основном тестовом наборе из 31 запроса этот флаг менял результат только для одного запроса — но и этого оказалось достаточно, чтобы прежний показатель 0,968 продолжал выглядеть как более высокий baseline.

Но затем обнаружилась ещё более неприятная деталь. Tenant boost сравнивал арендатора, найденного в запросе, со значением арендатора в metadata каждого фрагмента. А metadata в индексе к этому моменту уже не полностью соответствовали текущей базе документов. У **387 фрагментов** оставалось устаревшее значение.

Получилась довольно неприятная цепочка:



Рассинхронизация базы и индекса

То есть проблема была не просто в забытом флаге и не просто в устаревшем индексе. Способ улучшения поиска был основан на данных, которые к тому моменту перестали соответствовать содержимому базы.

После синхронизации данных, пересборки индекса и фиксации рабочей конфигурации воспроизводимым результатом стал **Recall@10 = 0,935**. Число меньше прежнего 0,968. Но оно намного полезнее, потому что я могу повторить эксперимент и получить то же состояние системы.

После этого вместе с метрикой я стал относиться к версии индекса и параметрам запуска как к части самого результата. Этот эпизод оказался для меня важнее многих экспериментов с моделями. Можно очень аккуратно измерять качество и при этом измерять не то состояние системы, которое реально используется.

В какой-то момент вопрос «как поднять Recall?» сменился другим:

Что именно я сейчас считаю правильным результатом и смогу ли воспроизвести его завтра?

Не каждый вопрос вообще должен попадать в RAG

Похожая проблема возникла со структурированными данными.

Допустим, пользователь спрашивает:

Кто не заплатил за май?

Можно найти документы о платежах, собрать контекст и дать его модели. Но если платежи уже лежат в таблице, это довольно странный путь.

Другой вопрос:

Какая ответственность предусмотрена за просрочку?

Здесь нужен договор.

А запрос:

Кто не заплатил за май и какая пеня предусмотрена его договором?

требует уже двух источников.

Так перед поиском появился слой маршрутизации:

Запрос → маршрут

- └─ SQL
- └─ RAG
- └─ SQL + RAG

Объяснить с 

Мне было важно сохранять не только выбранный маршрут, но и причину.

Например:

```
RouteDecision(  
    target="sql",  
    rule_name="payments_query",  
    matched_on="не заплатил",  
)
```

Объяснить с 

Если вопрос можно однозначно направить в нужный источник простым правилом, я предпочитаю правило ещё одному вызову LLM. На отдельном regression-наборе маршрутизация сейчас проходит **21 тест из 21**. Набор небольшой, поэтому это не доказательство идеальной

классификации. Его задача проще: если после очередного изменения знакомый тип запроса внезапно уйдёт не туда, тест это покажет.

Есть и резервный сценарий. Например, выбран SQL, но структурированный запрос ничего не вернул. Тогда система может продолжить поиск по документам. Отсюда появился следующий вопрос: если пользователь в итоге получил правильный ответ, как понять, каким путём система к нему пришла?

«Модель ошиблась» оказалось слишком неточным диагнозом

На раннем этапе я проверял систему самым естественным способом: задавал вопрос и смотрел на ответ.

- Хороший — отлично.
- Плохой — значит, надо менять промпт или модель.

Чем сложнее становился конвейер, тем бесполезнее становился такой подход. Ошибка могла произойти задолго до LLM.

- Маршрутизация могла выбрать неправильный источник.
- Поиск мог не найти нужный документ.
- Правильный документ мог найтись, но не попасть в top-k.
- Модель могла получить хороший контекст и неправильно его использовать.
- Или ответ мог быть правильным по смыслу, но содержать придуманную цифру.

Поэтому каждый запрос получил `trace_id`, а ключевые этапы обработки начали записываться:

- `trace_id`
- `route_initial`
- `route_final`
- `rule_name`
- `chunks`
- `model`
- `latency_ms`
- `guard_fired`

Если запрос первоначально ушёл в SQL, ничего не нашёл и затем переключился на RAG, в `trace` остаются оба маршрута. После этого фраза «LLM снова ошиблась» почти исчезла из отладки.

Вместо неё появилась последовательность вопросов:

- Правильно ли выбран источник?
- Найден ли нужный документ?
- Попал ли нужный фрагмент в контекст?

И только после этого:

- Что сделала с ним модель?

Трассировка сама по себе не повышает качество. Но без неё очень легко исправлять модель там, где модель вообще ни при чём.

Где я перестал доверять решения LLM

Самый важный архитектурный выбор появился уже не в поиске.

Предположим, в действующем договоре написано:

Пеня за просрочку оплаты — 0,05% за каждый день.

Контрагент предлагает:

Пеня — 0,01%.

Самое простое решение — передать старый и новый текст модели:

Сравни условия и оцени риски для арендодателя.

Современная LLM вполне способна написать убедительное объяснение. Меня не устраивала другая вещь: одно и то же изменение не должно сегодня считаться существенным, завтра умеренным, а после смены модели оцениваться ещё третьим способом. Поэтому я разделил извлечение фактов и принятие решения.

LLM приводит текст к структуре:

```
old_penalty = 0.05
new_penalty = 0.01
```

Объяснить с 

После валидации обычный код применяет правило. Модель при необходимости уже формулирует результат человеческим языком.

```
Текст
↓
Извлечение фактов
↓
Валидация
↓
Детерминированное правило
↓
Результат
↓
Формулировка ответа
```

Объяснить с 

Так появился принцип, который сейчас определяет существенную часть системы:

LLM извлекает факты и формулирует текст. Там, где решение можно выразить явным правилом, решение принимает код.

Это не попытка превратить право в набор if/else. Скорее наоборот: пришлось явно определить границы автоматизации. Если необходимых данных недостаточно или ситуация не покрывается playbook, система не должна достраивать решение только потому, что LLM способна продолжить

текст. Наверное, именно здесь юридический опыт повлиял на архитектуру сильнее всего. Основная сложность была не в написании промпта, а в определении того, какие факты действительно влияют на вывод, где существуют исключения и в какой момент автоматическая оценка перестаёт быть надёжной.

Отдельная проблема — цифры, которых не было

Для договоров некоторые виды галлюцинаций особенно неприятны. Модель может правильно передать общий смысл пункта, но ошибиться в сумме, сроке, проценте или номере документа.

Поэтому после генерации в системе работает отдельная проверка критичных числовых значений и реквизитов. Она сопоставляет их с тем, что действительно присутствовало в контексте или пришло из структурированного источника. Это не доказывает истинность ответа целиком и не заменяет оценку качества.

Но логика здесь та же: если конкретную проверку можно выполнить обычным кодом, нет причины спрашивать у той же модели, не придумала ли она что-нибудь.

Эксперимент, который я в итоге выбросил

Не каждое усложнение системы оказалось полезным. Один из примеров — reranker после объединения BM25 и векторного поиска. Гипотеза выглядела нормально: первый этап собирает кандидатов, cross-encoder точнее переставляет их по релевантности, качество растёт.

На моих данных произошло обратное.

Я попробовал два варианта: полную замену исходного ранжирования оценками cross-encoder и линейное смешивание с исходными score при $\alpha = 0,7$. В первом случае Recall@10 снизился с **0,935 до 0,806**, во втором — до **0,871**. Оба варианта оказались хуже базовой конфигурации.

Отдельно пришлось разбираться со скоростью. Инференс reranker занимал около **5,16 секунды** при установленном ограничении в **2,5 секунды**. После перехода на ONNX с квантизацией время удалось сократить до **2,45 секунды**. Проблему скорости я решил. Проблему качества — нет. Поэтому reranker в рабочем конвейере не остался.

Этот отрицательный результат оказался полезнее, чем ещё один компонент на архитектурной схеме. Вокруг RAG хватает техник, каждая из которых выглядит разумно: reranking, query rewriting, дополнительные LLM-проверки, агентный поиск. Но успешное применение метода в чужом проекте или его описание в статье ещё не гарантирует, что он улучшит поиск на других данных.

Поэтому новый компонент остаётся в системе только в том случае, если устраняет измеряемую проблему на моих данных.

Где система должна остановиться

Есть ситуации, в которых лучший результат — отсутствие автоматического вердикта.

- Если найден конкретный пункт договора, его можно показать и объяснить.
- Если есть явное правило, его можно воспроизводимо применить.
- Если данные находятся в таблице, их можно получить запросом.
- Но если фактов недостаточно, условия противоречат друг другу или случай не покрывается правилами, система должна это показать, а не компенсировать неопределённость уверенностью LLM.

В юридическом домене убедительный ответ и корректный ответ — не одно и то же. Иногда передача случая человеку — не ошибка автоматизации, а правильный результат.

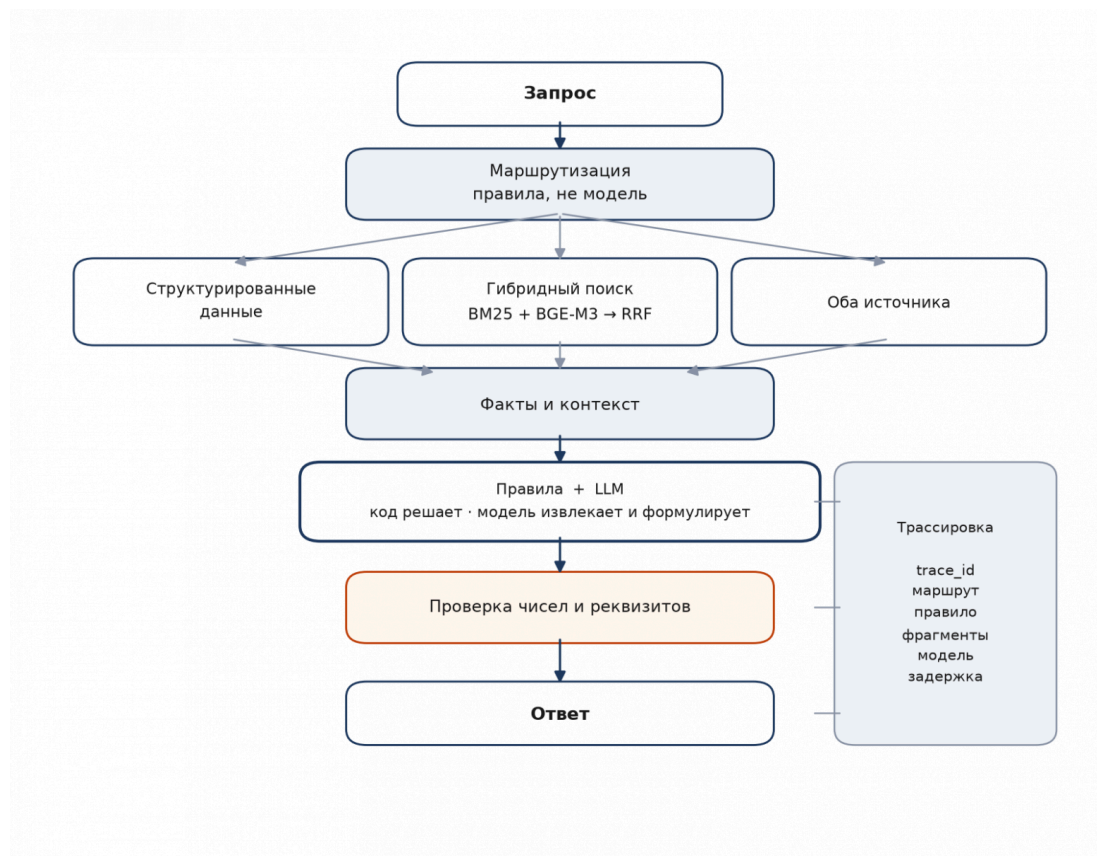
Что получилось в итоге

Первоначальная архитектура выглядела примерно так:

Вопрос → поиск → LLM → ответ

Объяснить с 

Сейчас она ближе к следующей:



Итоговая система

LLM в этой схеме по-прежнему выполняет работу, ради которой проект вообще имеет смысл: понимает свободный язык, извлекает факты из неструктурированного текста и формулирует понятный ответ. Но она больше не отвечает за всё подряд.

Если бы я начинал такой проект заново, я бы не стал сразу строить сложный RAG.

1. Сначала взял бы несколько десятков реальных вопросов.
2. Построил простейший baseline.
3. А затем смотрел бы не на то, какой ещё современный компонент можно добавить, а на то, **где именно система ошибается**.
 - Не находятся номера и фамилии — нужен лексический поиск.
 - Ответ уже есть в базе — не нужен RAG.
 - Нужно воспроизводимое решение — оно не должно целиком зависеть от LLM.
 - Непонятно происхождение ошибки — нужна трассировка.
 - Компонент не улучшает измеряемый результат — его можно удалить.

Самый неожиданный вывод из проекта для меня оказался вообще не про RAG. Получить от модели правдоподобный ответ сравнительно легко.

Гораздо сложнее определить, **какой ответ считать правильным, на каком состоянии данных он получен, можно ли его воспроизвести и почему система пришла именно к нему**.

Разница между 0,968 и 0,935 показала это лучше любого промпта. Первое число выглядело лучше. Второе оказалось полезнее. В какой-то момент главной задачей перестало быть сделать модель умнее. Гораздо важнее оказалось сделать всю систему проверяемой.

Архитектура, демонстрации и обезличенные примеры трассировки опубликованы. Код будет выложен на [GitHub](#) после завершения анонимизации данных: [репозиторий LegalRent](#).

Теги: RAG, юридические документы, LLM, гибридный поиск, BM25, BGE-M3, RRF, контроль галлюцинаций, legaltech

Хабы: Python, Искусственный интеллект, Data Engineering, Natural Language Processing, Машинное обучение



Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



4K+

1

Охват за 30 дней

Карма

Олег Мосийчук @OlegMos

Пользователь

Подписаться



Комментарии 5



ToxaBes

11 часов назад

Схема, к которой вы пришли уже стала классической схемой RAG-системы. К сожалению, она устарела уже почти на год.

В современной схеме для систем где критична точность (юр отдел, комплаенс, девелопмент) дополнительно используется:

1. контекстуальный чанкинг.
2. версионирование
3. дообучение самой модели под документы
4. дополнительные tools для модели, которые выходят за рамки комментария (diff по 2м документам и тд).

Получается система совсем другого уровня, которая не путает номера и пункты, ставки и даты, а также версии договоров.



Ответить



ENick

8 часов назад

""Способ улучшения поиска был основан на данных, которые к тому моменту перестали соответствовать содержимому базы."" за это студентам двойки ставят, а у юристов это такой высший пилотаж, что не стыдно такое всем показывать? Может сапоги тачать оставим сапожникам?

↑ ↓ Ответить

🔖 ...

○  **ToxaBes**
7 часов назад

^

Ну, человек сам создавал систему для себя, сам пришел к классической (немного устаревшей) схеме, по пути набил шишек, это нормально. А то, что он этим поделился, так вообще хорошо. Насколько я понял, автор не ML-инженер, а юрист. Я видел компании-интеграторы, где ML-инженеры делали хуже, поэтому, наверное, стоит "охлаждать трахание".

↑ ↓ Ответить

🔖 ...

○  **Ribera-sun**
7 часов назад

^

Видел много ваших ответов сегодня, возможно вы в теме и сможете ответить - насколько реально прямо сейчас перевести отность и powerbi в плоскость тестовых запросов обычных пользователей к модели, которая на лету будет собирать показывать цифры?

Условно от "сколько заявок с типом X было за год", до "сколько было заявок с типом X, на неделе Y от поставщика Z, которые были просрочены на сутки и более".

Причем перевести так, чтобы не приходилось после каждой пачки неверных ответов сажать живого аналитика и разбираться почему модель не смогла собрать данные, хотя у нее есть полное описание полей с синонимами, все формулы расчетов и прочее? Понимаю вопрос слишком кратко задан, но всё же.

↑ ↓ Ответить

🔖 ...

○  **ToxaBes**
6 часов назад

^

Причем перевести так, чтобы не приходилось после каждой пачки неверных ответов сажать живого аналитика и разбираться почему модель не смогла собрать данные, хотя у нее есть полное описание полей с синонимами, все формулы расчетов и прочее? Понимаю вопрос слишком кратко задан, но всё же.

Вы вполне понятно описали проблему. Еще год назад я бы сказал, что с приемлимой точностью это не сделать, но этот год бустанул вообще все сферы применения ML и эту тоже.

подавляющее большинство интеграторов решает эту задачу предоставляя модели прямой доступ к SQL, после чего получается ситуация подобная тому что описали вы. Я уже исправлял подобное в чужой ML-автоматизации, после того как интеграторов и след простыл.

Суть в том, что между LLM и базой данных ставится семантический слой (tools: metrics/dimensions/joins), определенный один раз заранее. Модель не пишет произвольный SQL к сырым таблицам, вместо этого она использует определенные функции (tools), передавая туда контекст (даты, интервалы, контрагенты) как параметры вызова функций.

Получив точные данные модель делает то для чего создана: обобщает, анализирует и предлагает выводы не придумывая цифры по различным сферам (заявки/поставщики/просрочка и тд).

При таком подходе результат детерминирован, а точность начинает осторожно щупать 100%. Но этот слой ML-инженеру нормально можно создать только в связке с самими аналитиками, делается это один раз и помогает избежать кучи граблей.

При таком подходе даже облачная фронтинг модель не нужна, достаточно локальной модели размером до ~30B. Разница между ними будет не в точности, а в скорости.

↑ ↓ Ответить

🔖 ...

Зарегистрируйтесь на Хабре, чтобы оставить комментарий